masai

AutoML for LLMs

Search Space Design • Bayesian Optimization • Early Stopping

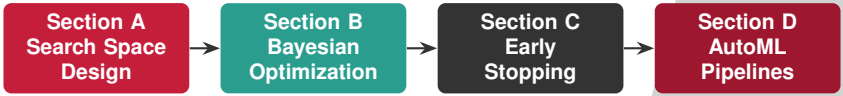
Priyansh Saxena

Masai School | IITP-AIML-2506 | Week 35 — Session 1/2

09 April 2026



Agenda



- 4 short quizzes after each section
- Case study: AutoML for a medical QA model
- Final cumulative quiz & wrap-up

masai

Why AutoML for LLMs Matters

The Problem

LLMs have **10+ tunable hyperparameters**: learning rate, LoRA rank, dropout, warmup, batch size, epochs. . .
Manual tuning: try 20 configs $\times$ 4 hours each = **80 GPU-hours** for ONE model-dataset pair.

The Solution

AutoML: 20 intelligent trials with pruning = **better result in 15 GPU-hours**.

- A team spent 2 weeks manually tuning — Optuna found a better config **overnight**
- Google, Meta, Microsoft all use automated HP optimization in production

masai

The Cost of Bad Hyperparameters

LR 10× too high → training diverges after 5 minutes (wasted run)

LoRA rank too low → model underfits, accuracy plateaus at 60%

No early stopping → discover the run was bad AFTER 6 hours

Cost Comparison

- 50 random trials × \$8/trial = **\$400 wasted**
- 20 Bayesian trials × \$3/trial = **\$60** for a *better* model

masai

From Grid Search to AutoML for LLMs



Key Insight

As models got more complex, manual tuning became impossible → automation became **necessary**.

masai

What Is a Search Space?

Definition

The set of **hyperparameters you choose to tune** and their **allowed ranges**. Defines WHERE the optimizer searches — *garbage in, garbage out*.

Two decisions:

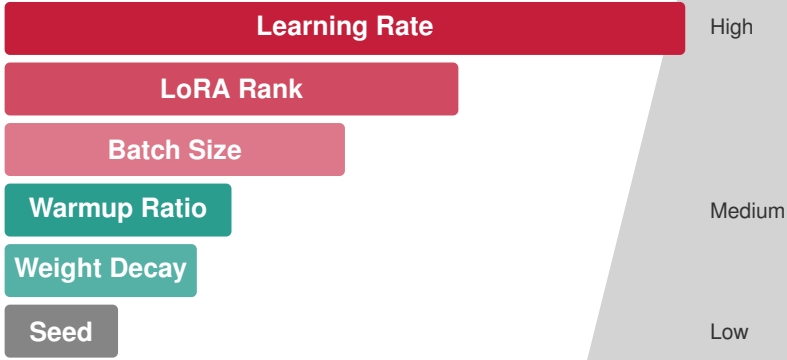
1. **Which parameters** to tune
2. **What range** for each

Example

```
search_space = {learning_rate: [1e-5, 5e-4], lora_rank: [4, 8, 16, 32]}
```

masai

Which Hyperparameters Matter?



Fix These (Don’t Tune)

Optimizer (AdamW), precision (bf16), max sequence length (task-dependent)

masai

Linear vs. Log Scale — Why It Matters

The Problem

Learning rate: good values span **1e-5 to 5e-4** (50× range). Linear uniform: 90% of samples between 1e-4 and 5e-4 — **starves the lower range**.

The Fix: Log Scale

Log uniform gives **equal probability to each order of magnitude** — balanced exploration.

Rule of Thumb

- **Log scale:** learning rate, weight decay, LoRA alpha
- **Linear / categorical:** integers (rank, batch size), ratios (warmup)

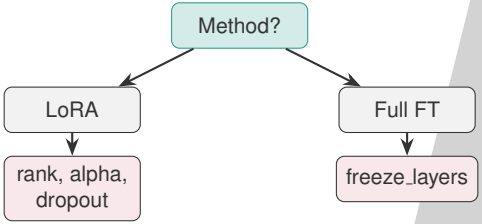
```
trial.suggest_float("lr", 1e-5, 5e-4, log=True)
```

masai

Conditional and Hierarchical Search Spaces

Key Idea

Some parameters only matter when others are set: LoRA alpha matters **only if using LoRA**, not full fine-tuning.



Optuna handles this with define-by-run: `if method == "lora": rank = trial.suggest_int(...)`

masai

Common Search Space Mistakes

Too Wide

`learning_rate [1e-8, 1.0]` — 99% of this range is useless for LLMs

Too Narrow

`learning_rate [2e-5, 3e-5]` — might miss the optimum entirely

Wrong Scale

Learning rate on linear scale — under-samples small values

Too Many Parameters

Tuning 15 things at once → curse of dimensionality

Result

Narrowing from 15 parameters to 5 reduced trials needed from 100 to **20**.

Quiz: Search Space Design

Q1. You're fine-tuning a 7B model with LoRA. Which HP is **MOST** important to tune?

(a) Random seed

(b) Learning rate

(c) Tokenizer vocabulary size

(d) Number of attention heads

masai

Quiz: Search Space Design

Q1. You're fine-tuning a 7B model with LoRA. Which HP is **MOST** important to tune?

(a) Random seed

(b) Learning rate

(c) Tokenizer vocabulary size

(d) Number of attention heads

Answer

(b) Learning rate — dominates performance; LoRA rank is second.

masai

Quiz: Search Space Design (cont.)

Q2. Why should learning rate be searched on a **log scale**?

(a) It's a convention

(b) Good learning rates span several orders of magnitude

(c) Log scale is faster to compute

(d) Linear scale doesn't work with Optuna

masai

Quiz: Search Space Design (cont.)

Q2. Why should learning rate be searched on a **log scale**?

(a) It's a convention

(b) Good learning rates span several orders of magnitude

(c) Log scale is faster to compute

(d) Linear scale doesn't work with Optuna

Answer

(b) Good LRs span 1e-5 to 5e-4 — log scale gives each order of magnitude equal probability.

Q3. True or False: Include as many hyperparameters as possible in your search space.

masai

Quiz: Search Space Design (cont.)

Q2. Why should learning rate be searched on a **log scale**?

(a) It's a convention

(b) Good learning rates span several orders of magnitude

(c) Log scale is faster to compute

(d) Linear scale doesn't work with Optuna

Answer

(b) Good LRs span 1e-5 to 5e-4 — log scale gives each order of magnitude equal probability.

Q3. True or False: Include as many hyperparameters as possible in your search space.

Answer

False. More parameters = curse of dimensionality. Tune only what matters.

masai

The Problem with Grid and Random Search



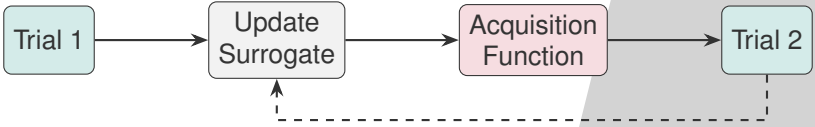
- Grid: 5 values $\times$ 5 $\times$ 5 = **125 trials** (500+ GPU-hours for LLMs)
- Random: better than grid, but still **doesn't learn** from past trials
- Bayesian: **clusters in the good region** after a few trials

masai

Bayesian Optimization — The Core Idea

Three Components

1. **Surrogate model** — probabilistic model of the objective function
2. **Update** — after each trial, update the surrogate with new results
3. **Acquisition function** — pick the next point: balance *exploitation* vs. *exploration*



masai

Surrogate Models: GP vs. TPE

Gaussian Process (GP)

- Models objective as a **smooth function**
- Excellent for $<$20 parameters
- Gives uncertainty estimates
- Scales poorly to high dims

TPE (Tree Parzen Estimators)

- Models **distribution of good vs. bad** configs
- Scales to 50+ parameters
- Handles mixed types, conditionals
- **Optuna's default**

For LLM Tuning

TPE is the default choice — handles mixed types, conditional spaces, and scales well. `optuna.create_study()` $\rightarrow$ uses TPE sampler automatically.

masai

Acquisition Functions: Exploit vs. Explore

Exploitation

Try configs **similar to the best** found so far

Exploration

Try configs where the model is **most uncertain**

Expected Improvement (EI)

"How much **better** could this point be than the current best?"

Balances both: high EI where predicted performance is good **AND** uncertainty is high.

Example

Trial 10: best loss = 0.35.

EI asks: "Could point X give loss < 0.35? How likely and by how much?"

masai

Optuna: Define-by-Run Bayesian Optimization

Key Features

- Define search space **inside** the objective function (not before)
- Supports conditional parameters naturally (just use if/else)
- Built-in visualization: optimization history, parameter importance

```python
import optuna

def objective(trial):
    lr = trial.suggest_float("learning_rate", 1e-5, 5e-4, log=True)
    lora_rank = trial.suggest_categorical("lora_rank", [4, 8, 16, 32])
    warmup = trial.suggest_float("warmup_ratio", 0.03, 0.1)
    model = train_lora_model(lr=lr, rank=lora_rank, warmup=warmup)
    return evaluate(model)

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=20)
```

masai

Optuna in Action — Convergence

Setup

- Task: fine-tune DistilBERT for sentiment classification
- Search space: learning_rate (log), batch_size (categorical), warmup_ratio (uniform)
- 20 trials with TPE sampler

Result

Bayesian optimization converges to optimal in **trial** $\sim$**12**.
Random search still exploring at trial 20.

Takeaway

20 Bayesian trials $\approx$ 60 random trials in final quality.

masai

Reading Optuna Results

Key Outputs

- `study.best_params` — the winning configuration
- `study.best_value` — the best validation metric
- **Parameter importance** — which parameters actually mattered?
- **Parallel coordinate plot** — relationships between params and objective

Example Result

```
learning_rate: 2.3e-4, lora_rank: 16, warmup: 0.06
→ val_loss: 0.31
```

Visualize with: `optuna.visualization.plot_optimization_history(study)`

masai

Quiz: Bayesian Optimization

Q1. What does Bayesian optimization use to decide the next config?

(a) Random dice

(b) A surrogate model that predicts performance from previous trials

(c) The gradient of the loss function

(d) A pre-defined grid

masai

Quiz: Bayesian Optimization

Q1. What does Bayesian optimization use to decide the next config?

(a) Random dice

(b) A surrogate model that predicts performance from previous trials

(c) The gradient of the loss function

(d) A pre-defined grid

Answer

(b) A surrogate model (TPE) that learns from previous trials.

masai

Quiz: Bayesian Optimization (cont.)

Q2. How is Bayesian optimization different from random search?

masai

Quiz: Bayesian Optimization (cont.)

Q2. How is Bayesian optimization different from random search?

Answer

It **learns from previous trials** to focus on promising regions — random search treats every trial independently.

Q3. In 20 trials, Bayesian optimization typically matches random search at how many trials?

(a) 20
(b) 50–100
(c) 200+
(d) They're always equivalent

masai

Quiz: Bayesian Optimization (cont.)

Q2. How is Bayesian optimization different from random search?

Answer

It **learns from previous trials** to focus on promising regions — random search treats every trial independently.

Q3. In 20 trials, Bayesian optimization typically matches random search at how many trials?

(a) 20
(b) 50–100
(c) 200+
(d) They're always equivalent

Answer

(b) 50–100 — Bayesian optimization is 3–5$\times$ more sample-efficient.

masai

Coming Up Next

Part 2: Early Stopping Strategies & AutoML Pipelines

masai

The Early Stopping Problem

Without Early Stopping

A bad hyperparameter config is often obvious after **10–20% of training**.
Without early stopping: you pay for 100% before discovering it's bad.

With Early Stopping

Kill bad trials early, reallocate compute to promising ones.

50 trials $\times$ 5 epochs each = 250 epochs total.
With pruning: $\sim$80 total epochs (**68% savings**).

masai

Patience-Based Early Stopping

How It Works

- Monitor validation loss every N steps
- If no improvement for **patience** evaluations $\rightarrow$ stop training
- Simple, effective — but only works **within a single trial**

```
from transformers import EarlyStoppingCallback

callback = EarlyStoppingCallback(
    early_stopping_patience=3,
    early_stopping_threshold=0.01
)
```

Validation loss curve: “3 checks left... 2... 1... **STOP**”

masai

Optuna Pruning: Early Stopping ACROSS Trials

Key Difference from Patience

Optuna compares **this trial against ALL previous trials** — not just its own history.

1. After each epoch, report metric: `trial.report(val_loss, epoch)`
2. Optuna compares against previous trials at the same step
3. If below median → **prune it** (`raise TrialPruned()`)

Example

Trial 15 at epoch 2 has val_loss = 0.8
Median of completed trials at epoch 2 was 0.5 → **PRUNE**

masai

Pruning Strategies: Median vs. Hyperband

MedianPruner

Prune if trial is **below median** of previous trials at same step.

$\sim$50% of trials survive each check.

PercentilePruner

Prune if below **top X%**.
Tunable aggressiveness.

HyperbandPruner

Multi-fidelity: runs **many trials for a few epochs**, keeps top fraction.

```
min_resource=1
max_resource=10
reduction_factor=3
```

Recommendation

For LLM fine-tuning: **HyperbandPruner** — aggressive early killing, focuses budget on winners.

masai

Setting Up Pruning in Optuna

```
def objective(trial):
    lr = trial.suggest_float("learning_rate", 1e-5, 5e-4, log=True)

    for epoch in range(10):
        train_loss = train_one_epoch(model, lr=lr)
        val_loss = evaluate(model)

        trial.report(val_loss, epoch)     # report metric
        if trial.should_prune():          # check pruning
            raise optuna.TrialPruned()

    return val_loss

study = optuna.create_study(
    direction="minimize",
    pruner=optuna.pruners.HyperbandPruner()
)
study.optimize(objective, n_trials=50)
```

masai

How Much Compute Does Pruning Save?

Typical Results

- 60–70% of trials are pruned after $<$30% of training
- 50 trials $\times$ 5 epochs: without pruning = 250 epoch-equivalents
- With pruning: $\sim$80 epoch-equivalents



masai

Quiz: Early Stopping

Q1. What does “patience” control in early stopping?

(a) How long to wait before starting training

(b) How many evaluations without improvement before stopping

(c) The maximum learning rate

(d) The number of warmup steps

masai

Quiz: Early Stopping

Q1. What does “patience” control in early stopping?

(a) How long to wait before starting training

(b) How many evaluations without improvement before stopping

(c) The maximum learning rate

(d) The number of warmup steps

Answer

(b) Number of evaluation steps without improvement before stopping the run.

masai

Quiz: Early Stopping (cont.)

Q2. A trial is pruned after 10% of training. How much compute was saved?

(a) 10%

(b) $\sim$50%

(c) $\sim$90%

(d) 0% — pruned trials still run to completion

masai

Quiz: Early Stopping (cont.)

Q2. A trial is pruned after 10% of training. How much compute was saved?

(a) 10%

(b) ∼50%

(c) ∼90%

(d) 0% — pruned trials still run to completion

Answer

(c) ∼90% — only 10% of the budget was used before pruning.

masai

The Full AutoML Pipeline



The Five Steps

1. **Search space** (Section A) — what and how wide to search
2. **Optimizer** (Section B) — Bayesian/TPE for intelligent search
3. **Pruning** (Section C) — kill bad trials early
4. **Optimization loop** — run trials, collect results
5. **Final model** — retrain with best params, full budget

masai

HuggingFace Trainer + Optuna Integration

Built-in Integration

`Trainer.hyperparameter_search()` handles everything:

- Optuna manages the search
- Trainer manages training
- Pruning via `HuggingFacePruningCallback`

```
best_run = trainer.hyperparameter_search(
    hp_space=hp_space,
    backend="optuna",
    n_trials=20,
    direction="minimize",
)
print(f"Best params: {best_run.hyperparameters}")
```

masai

Defining hp_space for HuggingFace

Key Rule

Function takes an Optuna trial, returns a dict.
Keys **must match** `TrainingArguments` parameter names.

```
def hp_space(trial):
    return {
        "learning_rate": trial.suggest_float(
            "lr", 1e-5, 5e-4, log=True),
        "per_device_train_batch_size": trial.suggest_categorical(
            "bs", [4, 8, 16]),
        "num_train_epochs": trial.suggest_int(
            "epochs", 1, 5),
        "warmup_ratio": trial.suggest_float(
            "warmup", 0.03, 0.1),
        "weight_decay": trial.suggest_float(
            "wd", 0.001, 0.1, log=True),
    }
```

masai

Weights & Biases Sweeps — An Alternative

W&B Sweeps

- YAML config defines search space
- Sweep agent runs trials automatically
- Supports Bayesian, random, and grid methods
- **Beautiful dashboards**: parallel coordinates, parameter importance, live curves

Pros

- Best visualization
- Easy setup
- Cloud-managed

Cons

- Less flexible than Optuna
- No built-in pruning
- Requires W&B account

masai

Comparing AutoML Tools for LLMs

Tool	Flexibility	Scalability	Ease of Use	Pruning
Optuna	High	Medium	Medium	Built-in
Ray Tune	High	High	Low	Via Optuna
W&B Sweeps	Medium	Medium	High	No
HF AutoTrain	Low	Medium	High	No

Recommendation

- **Starting out?** Use Optuna — most flexible, best pruning
- **Need distributed?** Ray Tune + Optuna backend
- **Want dashboards?** W&B Sweeps
- **Zero-code?** HuggingFace AutoTrain

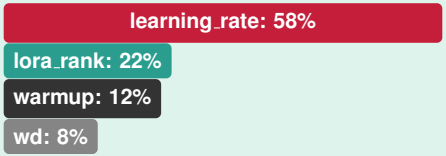
masai

Analyzing AutoML Results

Four Key Visualizations

1. **Optimization history** — which trials were best?
2. **Parameter importance** — which HPs mattered most?
3. **Parallel coordinates** — parameter correlations with performance
4. **Slice plots** — one parameter at a time vs. objective

Example Importance Breakdown



masai

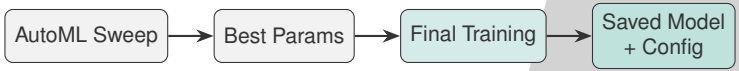
From Best Trial to Final Model

Important

The best trial gives you optimal hyperparameters — but that model was trained **during the sweep** (possibly with pruning).

Best Practice

1. **Extract** best params: `study.best_params`
2. **Retrain from scratch** with best params, full budget, no pruning
3. **Save** the model + log hyperparameters for reproducibility



masai

Quiz: AutoML Pipelines

Q1. Which library provides built-in `hyperparameter_search` with Optuna backend?

(a) PyTorch

(b) TensorFlow

(c) HuggingFace Transformers Trainer

(d) scikit-learn

masai

Quiz: AutoML Pipelines

Q1. Which library provides built-in `hyperparameter_search` with Optuna backend?

(a) PyTorch

(b) TensorFlow

(c) HuggingFace Transformers Trainer

(d) scikit-learn

Answer

(c) HuggingFace Transformers Trainer — one-line integration with Optuna.

masai

Quiz: AutoML Pipelines (cont.)

Q2. You ran 20 trials with pruning. 14 were pruned early. Good or bad?

masai

Quiz: AutoML Pipelines (cont.)

Q2. You ran 20 trials with pruning. 14 were pruned early. Good or bad?

Answer

Good! Compute was focused on the 6 promising trials.
14 bad configurations were killed early, saving 60–70% of budget.

masai

Case Study: AutoML for a Medical QA Model

Scenario

Healthcare startup fine-tuning **Llama-3-8B** for medical question answering.
Constraints: $500 GPU budget (A100s at $2/hr), 48-hour deadline.

Strategy

- **Search space:** LR (log), LoRA rank (4/8/16/32), alpha, dropout, warmup, batch size
- **Optimizer:** Optuna + HyperbandPruner, 30 trials
- **Pruning:** Kill after 20% of training

Result

Best config found in trial 18 → **87% accuracy** vs. 79% with default params. Total cost: $180 (well under $500 budget).

masai

Recap & Key Takeaways

1. **Search Space Design** — tune what matters (LR, LoRA rank), fix what doesn't, use log scale
2. **Bayesian Optimization** — learns from past trials; 20 Bayesian $\approx$ 100 random
3. **Early Stopping** — kill bad trials at 10–20% of training; save 40–68% compute
4. **AutoML Pipelines** — Optuna + HuggingFace Trainer = reproducible, hands-off optimization

masai

Final Quiz

Q: Fine-tune Llama-3-8B, budget for 20 runs. Which approach gives the best result?

(a) Grid search: 4 LRs $\times$ 5 ranks = 20 runs

(b) Random search: 20 random configs

(c) Bayesian optimization: 20 trials with pruning

(d) Manual tuning: 20 configs based on intuition

masai

Final Quiz

Q: Fine-tune Llama-3-8B, budget for 20 runs. Which approach gives the best result?

(a) Grid search: 4 LRs $\times$ 5 ranks = 20 runs

(b) Random search: 20 random configs

(c) Bayesian optimization: 20 trials with pruning

(d) Manual tuning: 20 configs based on intuition

Answer

(c) — Bayesian optimization with pruning gives the best config with the least waste.

masai



Thank You!

Questions?

Priyansh Saxena

All lecture materials available at:

priyanshsaxena.com/masai-iitp-aiml-2506